Experiment 9

Title

Implement search functionality to filter API results dynamically.

Theory:

Modern web applications often handle large amounts of data retrieved from external APIs. Displaying all fetched data at once can reduce usability. To improve user experience, search and filtering functionality is implemented to dynamically narrow down results based on user input.

An API (Application Programming Interface) provides structured data in formats such as JSON. JavaScript fetches this data asynchronously and stores it in memory. A search input field allows users to enter keywords, which are used to filter the API results in real time.

1. **Dynamic Search Filtering**

Dynamic search filtering allows data to be filtered without refreshing the page. As the user types in the search box, JavaScript compares the input text with the fetched data and displays only matching records.

2. **AJAX and Fetch API**

The Fetch API is used to retrieve data asynchronously from the server. It returns a promise that resolves with the API response, which is then converted into JSON format. This ensures smooth and fast data retrieval.

3. **JavaScript Array Methods**

JavaScript provides built-in array methods such as filter() and includes() to efficiently search through data. These methods help compare user input with data fields like name, title, or description.

4. **DOM Manipulation**

The filtered data is displayed dynamically using DOM manipulation, ensuring instant updates to the user interface whenever the search input changes.

Algorithm / Procedure

1. Start the experiment.
2. Create an HTML file with a search input field.
3. Use JavaScript to fetch data from a public API.
4. Store the API response in an array.
5. Display all API results on the webpage.
6. Capture user input from the search field.
7. Filter the stored data based on the search keyword.
8. Update the displayed results dynamically.
9. Repeat filtering as the user types.
10. Stop the experiment.

Program / Code:

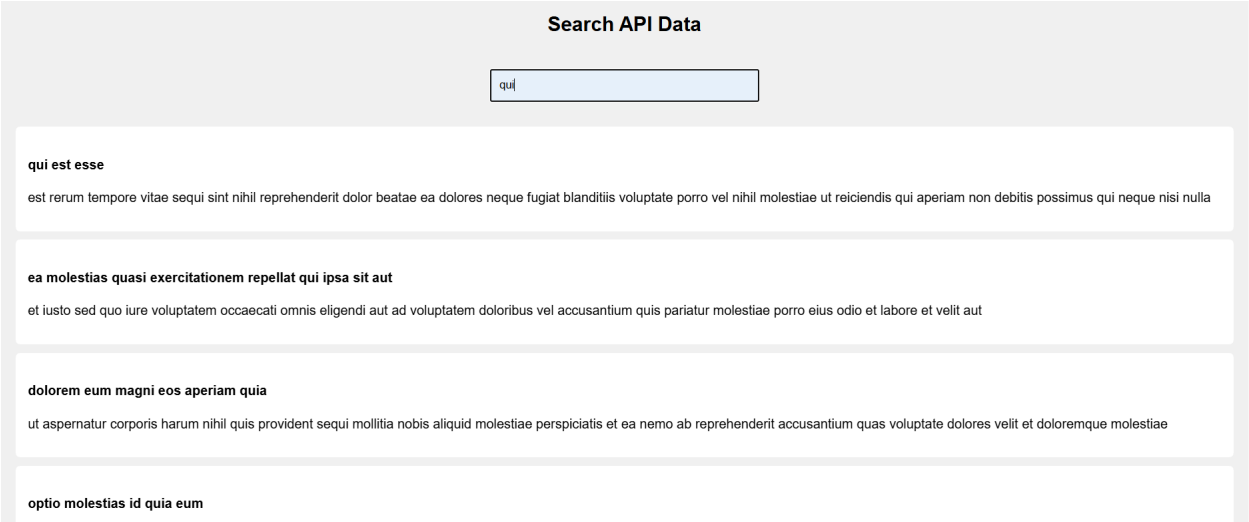
```
<!DOCTYPE html>
<html>
<head>
  <title>API Search Filter</title>
  <style>
    body {
      font-family: Arial;
      background: #f4f4f4;
    }
    input {
      width: 300px;
      padding: 10px;
      margin: 20px;
```

```
        }
        .card {
            background: white;
            padding: 15px;
            margin: 10px;
            border-radius: 5px;
        }
    </style>
</head>
<body>

<h2 align="center">Search API Data</h2>
<center>
<input type="text" id="search" placeholder="Search by title...">
</center>

<div id="results"></div>

<script>
let apiData = [];

fetch("https://jsonplaceholder.typicode.com/posts")
.then(response => response.json())
.then(data => {
    apiData = data;
    displayData(apiData);
```

```
});

function displayData(data) {
    const resultDiv = document.getElementById("results");
    resultDiv.innerHTML = "";

    data.forEach(item => {
        resultDiv.innerHTML += `
            <div class="card">
                <h4>${item.title}</h4>
                <p>${item.body}</p>
            </div>
        `;
    });
}

document.getElementById("search").addEventListener("keyup", function() {
    const keyword = this.value.toLowerCase();
    const filteredData = apiData.filter(item =>
        item.title.toLowerCase().includes(keyword)
    );
    displayData(filteredData);
});
</script>

</body>
```

</html>

Output:



Conclusion

Thus, dynamic search functionality was successfully implemented to filter API results using JavaScript. This experiment demonstrates the use of Fetch API, array filtering methods, and DOM manipulation to create responsive and interactive web applications.

Question:

1.What is dynamic search filtering?

Dynamic search filtering is a UI feature that updates a list of items in real-time as you type, instantly hiding non-matching results.

2.How is DOM manipulation used in this experiment?

DOM manipulation is used to show, hide, or update specific HTML elements (like list items) based on the user's search input without refreshing the entire page.

3. Why is page reload not required?

Page reload is not required because JavaScript updates only the specific parts of the Virtual DOM (or the real DOM) that need to change, keeping the application's current state and focus intact.